A BEGINNER-FRIENDLY APPLICATION MAP

How the INT ABCPA application works

Follow a request from the person using the site through the app and API to the place that stores its data.

1. People

Visitors browse the site, sign in, book a show, or update CMS content.

Website visitor

Content editor

→

2. React app

Public pages: home, Showtime, About, Pulse & Page.

Other pages: login, account, seat selection, demo checkout.

CMS: About, Pulse, and website-section editors.

→

3. Express API

Receives content requests, applies app rules, and sends back data.

/api/about

/api/pulse

/api/sections

/api/uploads

→

4. Data & files

MongoDB: About, Pulse posts, and site-section content.

Backend files: uploaded images and videos.

Database

Media files

Saved content and file URLs come back through the API and are shown by the React pages.

Sign-in uses a separate service

Login and signup use Firebase Authentication. The app then shows the user's account pages.

Accounts and demo bookings

Profile, addresses, and bookings are kept in this browser's localStorage—not in MongoDB. Checkout is a demo and does not charge a payment.

Newsletter & CMS access

Insider signup currently updates the page only. The CMS routes do not currently require a login gate.

A CLOSER LOOK AT CONTENT MANAGEMENT

CMS, API & database workflow

The same content API is used to load and save CMS content, and to show it on public pages.

1. **Open an editor**
   Choose About, Pulse, or a page section such as Hero or Showtime.
2. **Load content**
   React asks the API for the current content using a GET request.
3. **Edit & upload**
   Change text and links. Upload an image or video when needed.
4. **Save content**
   The API receives the changes and saves the content record.
5. **Show on the site**
   Public pages request the saved content and render it for visitors.

When an editor adds media

Select image or video → Send file to `/api/uploads/image` or `/api/uploads/video`

Backend checks the file → Saves it in `backend/uploads`

API returns a media URL → CMS saves that URL with the content

Public page loads the URL → Browser displays the image or video

Where content is stored

MongoDB connected: content is saved in MongoDB collections.

MongoDB unavailable: backend uses temporary in-memory content; restart clears it.

API unavailable in browser: some editors fall back to this browser's localStorage.

Important: browser fallback data does not sync to MongoDB or other devices.

Local development: Vite forwards `/api` and `/uploads` requests to the Express server. Production hosting must route those same paths to the backend.